

SP3O: Reinforcement Learning from Segment Preferences without Reward Modeling

Evan Assmus
University of Michigan, Ann Arbor
eassmus@umich.edu

Qining Zhang
University of Michigan, Ann Arbor
qiningz@umich.edu

Lei Ying
University of Michigan, Ann Arbor
leiyiing@umich.edu

Abstract

Preference-based reinforcement learning (PbRL) for general stochastic MDPs often requires training a reward model. Existing reward-model-free methods are either restricted to bandits or deterministic MDPs, such as DPO or P3O, or use zeroth-order, gradient-free optimization, which in general exhibits a slower convergence rate than gradient-based algorithms. Furthermore, existing reward-model-free preference-based RL algorithms almost exclusively use trajectory-level feedback, which can require significant effort from a human evaluator when trajectories are long. On the other hand, segments are much shorter, so they are easier to compare and evaluate. In this paper, we introduce a novel reward-model-free, critic-free, and gradient-based PbRL algorithm compatible with segment preferences named Segment Pairwise Proximal Policy Optimization (SP3O). SP3O utilizes segment-level preference feedback to construct an accurate policy value difference estimator via off-policy importance sampling, and then uses the estimator to compute the policy gradient via a PPO-type loss function. We provide a theoretical basis for the algorithm and analyze the tradeoff in choosing the segment length. We also evaluate it experimentally against other PbRL/RLHF algorithms in robotic control and LLM finetuning settings to show its improved performance, especially in long-horizon tasks.

1 Introduction

Reinforcement Learning (RL) (Sutton et al., 1998) has seen great success across the domains of game playing (Silver et al., 2016; Vinyals et al., 2019; Mnih et al., 2013) and robotic control (Schulman et al., 2017; Haarnoja et al., 2018). However, applying RL to more complex real-world tasks remains challenging, where one major bottleneck is designing a reward function that will naturally encourage agents to learn the desired behavior and avoid reward hacking (Skalse et al., 2022). Preference-based RL (PbRL) has emerged as a powerful framework for addressing these issues, in which, instead of receiving a numerical reward at each interaction step, the agent actively queries preferences over behaviors from oracles (humans or AI systems (Lee et al., 2023a)). The candidates to be compared usually take the form of pairs of full trajectories or segments. The oracle then returns its preferences about the candidates, which are then used to improve and optimize the policy. This method has seen great success in the domains of fine-tuning language models (Ouyang et al., 2022; Bai et al., 2022; Rafailov et al., 2023; Shao et al., 2024), continuous control tasks (Christiano et al., 2017), game playing (Christiano et al., 2017; Ibarz et al., 2018), and fine-tuning image generation models (Lee et al., 2023b).

Reward Model. Many existing preference-based RL algorithms, including Reinforcement Learning from Human Feedback (RLHF) for LLMs (Ouyang et al., 2022; Bai et al., 2022; Shao et al., 2024), work by training an

additional neural network, called the *reward model*, and using a classic reward-based RL method, such as PPO (Schulman et al., 2017), to optimize the policy (Christiano et al., 2017) from the numeric reward generated by the reward model. Depending on the application, the reward model can be trained offline, online, or a combination of both. The process generally assumes that preferences come from a Bradley-Terry model (Bradley and Terry, 1952) based on an unknown true reward function, and then maximizes the log-likelihood of seeing the preferences. Specifically, most of these works assume that preference processes are determined by either the partial reward sum of a trajectory segment (Christiano et al., 2017; Ibarz et al., 2018) or the total reward of a trajectory (Ouyang et al., 2022; Bai et al., 2022), i.e., for two trajectories (segments) σ^1 and σ^2 , both consisting of L steps, the probability of preferring one candidate over the other is governed by the partial return model, i.e.,

$$\mathbb{P}[\sigma^1 > \sigma^2] = \text{logistic} \left(\sum_{k=1}^L r(s_k^1, a_k^1) - \sum_{k=1}^L r(s_k^2, a_k^2) \right), \quad (1)$$

where $\sigma^1 = \{s_k^1, a_k^1\}_{k=1}^L$ and $\sigma^2 = \{s_k^2, a_k^2\}_{k=1}^L$ are the states and actions in time sequence constituting trajectories (segments) σ^1 and σ^2 respectively, and $\text{logistic}(\cdot)$ is the standard logistic function. One may also replace the logistic function with other link functions (Zhang and Ying, 2025a). This formulation has been successful in practice, but recent works raise questions about the adequacy of such a simple preference model, and variants have also been proposed (Knox et al., 2022; Hejna et al., 2023). More importantly, training a reward model is challenging: it complicates the training pipeline, requires additional memory and computational resources, and raises new concerns during evaluation, including distribution shift, lack of ground truth, and overfitting in joint training (Casper et al., 2023). Therefore, it is usually more appropriate to treat reward models as semi-quantitative evaluation tools instead of a perfect proxy of the true reward function.

RL Directly from Preferences. Methods that do not rely on reward model training, such as DPO (Rafailov et al., 2023) and P3O (Wu et al., 2024), have also been introduced, but only under restrictive assumptions. For example, these algorithms were originally designed for contextual bandits and can only be generalized to MDPs with a deterministic transition, making them unsuitable for more complex real-world RL problems where stochasticity naturally arises from both environments and the interaction with humans. Furthermore, these algorithms require preferences to be between full trajectories that start from the same state. For general RL settings, one approach is to use zeroth-order methods. For example, Zhang and Ying (2025a,b) introduced a zeroth-order policy optimization algorithm that works by perturbing and comparing two policies to perform a zeroth-order policy update. Another approach is to use an evolutionary optimization strategy with preferences (Busa-Fekete et al., 2014). However, in optimization, it is known that zeroth-order and evolutionary approaches usually do not converge as fast as gradient-based first-order methods, especially when the size of the actor neural network scales up.

RLHF with Segments. Evaluating short segments instead of long trajectories is often easier for human evaluators. Furthermore, when preferences are binary, evaluating more short segments gives more information than evaluating long segments even when the total length is the same. For these reasons algorithms have been developed that use preferences between segments (Wilson et al., 2012; Christiano et al., 2017; Lai et al., 2024; Guo et al., 2026). However, these methods either require segments that start from the same state (Lai et al., 2024; Wilson et al., 2012; Guo et al., 2026) which is not possible in some environments or use a reward model (Christiano et al., 2017; Guo et al., 2026).

In this paper, we consider a general stochastic MDP and aim to design a gradient-based algorithm for preference-based RL without reward modeling. Furthermore, for reasons discussed above, we are interested in RL with segment preferences, where segments do not necessarily start from the same state. In short, our goal is to train a policy directly from preferences between arbitrary segments.

1.1 Main Contributions

In this paper, we introduce a novel PbRL algorithm using segment preferences without reward models, called *Segment Pairwise Proximal Policy Optimization* (SP3O). SP3O can be viewed as a generalization of P3O (Wu et al., 2024) to the stochastic MDP setting with segment preferences and allowing different starting states for preferences.

Algorithm Design. SP3O has the following characteristics:

- SP3O is reward-model-free, critic-free, and applicable to general stochastic MDPs.
- SP3O uses segment-level preferences instead of trajectory-level preferences, which are more efficient to evaluate.
- SP3O uses gradient-based policy optimization and a hybrid on-policy and off-policy approach to make efficient use of human preferences.

Theoretical Analysis. We provide theoretical backup for the design of SP3O:

- We show that RL with segment feedback can be viewed as a discounted finite-horizon MDP, with carefully defined terminal rewards and initial state distribution, and prove that the policy gradient of this finite-horizon MDP is a scaled version of the policy gradient of the original MDP (Theorem 1). This formally justifies that policy gradient can be computed with segment preferences.
- We characterize an estimator error bound for a given segment length (Proposition 1), which reveals a fundamental tradeoff in choosing the segment length, which should not be too small or too large.

Experimental Evaluations. We evaluate SP3O empirically against Online DPO (Guo et al., 2024), P3O (Wu et al., 2024), and ZPG (Zhang and Ying, 2025a) in simulated robotic control environments and an LLM finetuning task. In both experiments we show that SP3O outperforms the other algorithms in most settings, with its advantage growing as the horizon grows. We also conduct a small ablation study on the segment length to show that using either a too small or a too large segment length can result in sub-optimal performance, which is consistent with the tradeoff described in Proposition 1.

2 Preliminaries

We consider an infinite-horizon discounted Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, d_0, r, \gamma, P)$, with state space $\mathcal{S}$, action space $\mathcal{A}$, initial state distribution d_0 , reward function $r(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, discount factor $\gamma \in (0, 1)$, and transition kernel P . We consider the reinforcement learning from preference feedback setting, where the agent does not directly observe reward $r(s, a)$ and has to be trained with preference feedback usually generated by humans.

A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is a function from the state space to a probability distribution on actions. We consider a parametrized policy π_θ , where θ could be a neural network. We define the state and state-action value functions for MDP $\mathcal{M}$ under policy π as:

$$V_{\mathcal{M}}^\pi(s) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t), s_1=s} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) \right], \quad (2a)$$

$$Q_{\mathcal{M}}^\pi(s, a) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t), s_1=s, a_1=a} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) \right], \quad (2b)$$

where s_t is the state at time t and a_t is the action taken at time t . We also define the state occupancy measure of policy π at timestep t as $d_{\mathcal{M},t}^\pi(s) = \mathbb{P}[s_t = s \mid s_1 \sim d_0, \pi]$, the discounted state occupancy measure of policy π as: $d_{\mathcal{M}}^\pi(s) = (1 - \gamma) \sum_{t=1}^{\infty} \gamma^{t-1} d_{\mathcal{M},t}^\pi(s)$, and the value of policy π as the expected discounted reward as follows,

$$J_{\mathcal{M}}(\pi) = \mathbb{E}_{s_1 \sim d_0}[V_{\mathcal{M}}^\pi(s_1)] \quad (3)$$

Our goal is to find policy π_{θ^*} that maximizes the value, i.e., $\theta^* \in \arg \max_{\theta} J_{\mathcal{M}}(\pi_{\theta})$.

2.1 Preference Feedback on Segments

Given two segments $\sigma^1 = \{s_k^1, a_k^1\}_{k=1}^L$ and $\sigma^2 = \{s_k^2, a_k^2\}_{k=1}^L$ with length L , we assume σ^1 is preferred over σ^2 with probability:

$$\mathbb{P}[\sigma^1 > \sigma^2] = \text{logistic}\left(\left[R(\sigma^1) + \gamma^{L-1}\hat{Q}(s_L^1, a_L^1)\right] - \left[R(\sigma^2) + \gamma^{L-1}\hat{Q}(s_L^2, a_L^2)\right]\right), \quad (4)$$

where $R(\sigma) = \sum_{k=1}^{L-1} \gamma^{k-1} r(s_k, a_k)$ is the discounted reward sum of segment σ with length L and

$$\hat{Q}(s_L, a_L) = Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_L, a_L) + \xi$$

is a noisy estimation of $Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_L, a_L)$ (the Q -value of the last state-action pair of the reference policy) and $\xi \in [-\nu, \nu]$ is a bounded noise that could be adversarial.

Given an estimate of the preference $\hat{\mathbb{P}}[\sigma^1 > \sigma^2]$, we can use the inverse of the logistic function to recover an estimate $D(\sigma^1, \sigma^2)$ of the human-perceived return difference:

$$D(\sigma^1, \sigma^2) = \text{logistic}^{-1}(\hat{\mathbb{P}}[\sigma^1 > \sigma^2]) \approx \left[R(\sigma^1) + \gamma^{L-1}\hat{Q}(s_L^1, a_L^1)\right] - \left[R(\sigma^2) + \gamma^{L-1}\hat{Q}(s_L^2, a_L^2)\right]. \quad (5)$$

We note that even though the partial return preference model in equation 1 has been assumed in most of the PbRL/RLHF literature (Christiano et al., 2017; Sadigh et al., 2017; Ibarz et al., 2018; Lee et al., 2021; Ouyang et al., 2022; Bai et al., 2022), it has been observed that the real preference of humans also depends on the quality of the final state-action pair (Knox et al., 2022) of the segment, which is not captured in the partial return model equation 1. To address this Knox et al. (2022) introduced the regret model

$$\mathbb{P}_{\text{regret}}[\sigma^1 > \sigma^2] = \frac{\exp\left(\sum_{t=1}^L -A^{\pi^*}(s_t^1, a_t^1)\right)}{\exp\left(\sum_{t=1}^L -A^{\pi^*}(s_t^1, a_t^1)\right) + \exp\left(\sum_{t=1}^L -A^{\pi^*}(s_t^2, a_t^2)\right)},$$

where A^{π^*} is the advantage function of the optimal policy. However, it is possible that the optimal policy is unknown to the human, meaning that they cannot meaningfully estimate this quantity. In contrast, our model only requires the human to be able to approximate the future performance of the policy they are currently viewing, which allows them to extrapolate viewed behavior to future states. For instance, in a goal-reaching task an evaluator would prefer a segment where the agent is moving closer to the goal over a segment where the agent is moving away, even if the average distance to the goal is the same. The partial return model may not capture this difference, while ours can. We provide a further comparison of preference models in Section A.

3 Segment Pairwise Proximal Policy Optimization

In this section, we first present our algorithm, SP3O (Algorithm 1), based on segment preferences. Each policy iteration of SP3O consists the following steps:

- sample R trajectories from a reference policy $\pi_{\theta_{\text{ref}}}$;
- use these trajectories to sample N_p segment pairs $\{(\sigma_n^1, \sigma_n^2)\}_{n=1}^{N_p}$ with a discounted probability distribution (Algorithm 2);
- use the preference to generate estimates of the return difference $D(\sigma_n^1, \sigma_n^2)$ for each segment pair and create a preference dataset $\mathcal{D}_{\text{ref}} = \{(\sigma_n^1, \sigma_n^2, D(\sigma_n^1, \sigma_n^2))\}_{n=1}^{N_p}$;
- estimate the policy gradient from a loss function (Equation 6) constructed through off-policy importance sampling, and update the current policy π_{θ_t} accordingly.

Algorithm 1 Segment Pairwise Proximal Policy Optimization (SP3O)

Require: initial parameter θ_0 , trajectory segment length L , preference oracle, number of policy updates B , number of policy updates per reference policy T , effective horizon $H = C_0/(1 - \gamma)$ which is a multiple of L , number of reference trajectories R , number of trajectory pairs N_p , number of human evaluators M ;

- 1: $\theta_{\text{ref}} \leftarrow \theta_0$;
 - 2: **for** $i = 1 : B$ **do**
 - 3: sample a batch $\mathcal{T}_{\text{ref},i} = \{\tau_j\}_{j=1}^R$ of trajectories with length H using behavior policy $\pi_{\theta_{\text{ref}}}$;
 - 4: sample segment pairs $\{(\sigma_n^1, \sigma_n^2)\}_{n=1}^{N_p}$ with Algorithm 2;
 - 5: **for** $n = 1 : N_p$ **do**
 - 6: query the preference oracle M times and gather the binary feedback $\{o_{n,m}\}_{m=1}^M$ where
 - 7: $\mathbb{P}[o_{n,m} = 1] = \mathbb{P}[\sigma_n^1 > \sigma_n^2]$ and estimate the preference as $\hat{\mathbb{P}}[\sigma_n^1 > \sigma_n^2]$ as $\frac{\sum_{m=1}^M o_{n,m}}{M}$;
 - 8: for each segment pair (σ_n^1, σ_n^2) , use $\hat{\mathbb{P}}[\sigma^1 > \sigma^2]$ to obtain $D(\sigma_n^1, \sigma_n^2)$ using Equation 5 ;
 - 9: create preference dataset $\mathcal{D}_{\text{ref}} = \{(\sigma_n^1, \sigma_n^2, D(\sigma_n^1, \sigma_n^2))\}_{n=1}^{N_p}$;
 - 10: **for** $t = 1 : T$ **do**
 - 11: update θ_t by minimizing $\mathcal{L}_{\text{SP3O}}(\theta_t; \theta_{t-1}, \theta_{\text{ref}}, \mathcal{D}_{\text{ref},i})$ equation 6;
 - 12: $\theta_{\text{ref}} \leftarrow \theta_T, \theta_0 \leftarrow \theta_{\text{ref}}$;
 - 13: **return** θ_T
-

Algorithm 2 Sample Segment Pairs

Require: trajectories $\{\tau_j\}_{j=1}^R$ with $\tau_j = \{s_{j,t}, a_{j,t}\}_{t=1}^H$, segment length L , number of pairs N_p ;

- 1: decompose each τ_j into $\{(s_{j,t}, a_{j,t})\}_{t=1}^H$;
 - 2: split trajectories into non-overlapping segments $\psi_{i,j} = \{(s_{i,jL+t}, a_{i,jL+t})\}_{t=1}^L$ for $1 \leq i \leq R$ and $0 \leq j < H/L$;
 - 3: assign each segment $\psi_{i,j}$ a weight $w_{i,j} \leftarrow \gamma^{L \cdot j}$;
 - 4: sample first segments $\{\sigma_n^1\}_{n=1}^{N_p}$ from $\psi_{i,j}$ s according to their weights such that $\psi_{i,j}$ is sampled with probability $\frac{w_{i,j}}{\sum_{i',j'} w_{i',j'}}$;
 - 5: sample second segments $\{\sigma_n^2\}_{n=1}^{N_p}$ from $\psi_{i,j}$ s according to their weights such that $\psi_{i,j}$ is sampled with probability $\frac{w_{i,j}}{\sum_{i',j'} w_{i',j'}}$ ensuring that $\sigma_n^1 \neq \sigma_n^2$;
 - 6: **return** $\{(\sigma_n^1, \sigma_n^2)\}_{n=1}^{N_p}$
-

Our algorithm uses a loss function with PPO-style clipping regularization defined below

$$\mathcal{L}_{\text{SP3O}}(\theta_t; \theta_{t-1}, \theta_{\text{ref}}, \mathcal{D}_{\text{ref}}) = -\frac{1}{N_p} \sum_{n=1}^{N_p} \min \left(D(\sigma_n^1, \sigma_n^2) \prod_{k=1}^L \frac{\pi_{\theta_t}(a_{n,k}^1 | s_{n,k}^1)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^1 | s_{n,k}^1)} \frac{\pi_{\theta_{t-1}}(a_{n,k}^2 | s_{n,k}^2)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^2 | s_{n,k}^2)}, \right. \\ \left. D(\sigma_n^1, \sigma_n^2) \prod_{k=1}^L \left(\frac{\pi_{\theta_t}(a_{n,k}^1 | s_{n,k}^1)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^1 | s_{n,k}^1)} \right)_{1-\epsilon}^{1+\epsilon} \left(\frac{\pi_{\theta_{t-1}}(a_{n,k}^2 | s_{n,k}^2)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^2 | s_{n,k}^2)} \right)_{1-\epsilon}^{1+\epsilon} \right) \quad (6)$$

where $(\cdot)_{1-\epsilon}^{1+\epsilon}$ is the clip operator in PPO, i.e., $(\cdot)_{1-\epsilon}^{1+\epsilon} = \min\{\max\{\cdot, 1 - \epsilon\}, 1 + \epsilon\}$.

We next explain the construction of this loss function. At policy iteration step t , the policy gradient can be computed using the policy value difference since:

$$\nabla_{\theta_t} J_{\mathcal{M}}(\pi_{\theta_t}) = \nabla_{\theta_t} (J_{\mathcal{M}}(\pi_{\theta_t}) - J_{\mathcal{M}}(\pi_{\theta_{t-1}})).$$

The policy value difference $J_{\mathcal{M}}(\pi_{\theta_t}) - J_{\mathcal{M}}(\pi_{\theta_{t-1}})$ can be evaluated *off-policy* using trajectories generated from the reference policy π_{ref} of length $H = C_0/(1 - \gamma)$ according to the following lemma.

Lemma 1 Suppose we have three policies $\pi_1, \pi_2, \pi_{\text{ref}}$. Assume that the support of π_1 and π_2 is enclosed within the support of π_{ref} , then a surrogate to estimate $J_{\mathcal{M}}(\pi_1) - J_{\mathcal{M}}(\pi_2)$ is given by the following approximation:

$$J_{\mathcal{M}}(\pi_1) - J_{\mathcal{M}}(\pi_2) \\ = \mathbb{E}_{\substack{a_t^1, a_t^2 \sim \pi_{\text{ref}}, \\ s_1^1, s_1^2 \sim d_0}} \left[\prod_{t=1}^H \frac{\pi_1(a_t^1 | s_t^1)}{\pi_{\text{ref}}(a_t^1 | s_t^1)} \frac{\pi_2(a_t^2 | s_t^2)}{\pi_{\text{ref}}(a_t^2 | s_t^2)} \times \left(\sum_{t=1}^H \gamma^{t-1} r(s_t^1, a_t^1) - \sum_{t=1}^H \gamma^{t-1} r(s_t^2, a_t^2) \right) \right] + \lambda, \quad (7)$$

where $\lambda = \mathcal{O}(\gamma^H) = \mathcal{O}(e^{-C_0})$ due to the use of an effective horizon and $\tau^1 = \{(s_t^1, a_t^1)\}_{t=1}^H$ and $\tau^2 = \{(s_t^2, a_t^2)\}_{t=1}^H$ are independent trajectories.

The proof of this lemma can be found in Section D.1. Since the agent does not directly observe the reward $r(s_t, a_t)$, the algorithm relies on preference feedback. In particular, we use $D(\tau^1, \tau^2)$ as an approximation of $\sum_{t=1}^H \gamma^{t-1} r(s_t^1, a_t^1) - \sum_{t=1}^H \gamma^{t-1} r(s_t^2, a_t^2)$. Note that our algorithm uses segments of length L instead of full trajectories with length H because a comparison of long trajectories could be time-consuming for the evaluators. For example, in autonomous driving, it is hard for a human evaluator to compare trajectories that are hours-long. In Section 4, we show that under mild assumptions, the policy gradient estimated from segment preferences is the same as the policy gradient estimated from full trajectory preferences, up to a constant scaling factor.

Now, given a set of segment pairs generated by the reference policy $\{(\sigma_n^1, \sigma_n^2)\}_{n=1}^{N_p}$, where $\sigma_n^1 = \{(s_{n,k}^1, a_{n,k}^1)\}_{k=1}^L$ and $\sigma_n^2 = \{(s_{n,k}^2, a_{n,k}^2)\}_{k=1}^L$, we can use the preference dataset to estimate the policy difference as:

$$J_{\mathcal{M}}(\pi_{\theta_t}) - J_{\mathcal{M}}(\pi_{\theta_{t-1}}) \approx \frac{1}{N_p} \sum_{n=1}^{N_p} D(\sigma_n^1, \sigma_n^2) \prod_{k=1}^L \frac{\pi_{\theta_t}(a_{n,k}^1 | s_{n,k}^1)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^1 | s_{n,k}^1)} \frac{\pi_{\theta_{t-1}}(a_{n,k}^2 | s_{n,k}^2)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^2 | s_{n,k}^2)},$$

without using any trajectories generated by π_{θ_t} or $\pi_{\theta_{t-1}}$. To further facilitate stability, we plug this estimator into the PPO-type clip mechanism (Schulman et al., 2017; Wu et al., 2024), which leads to the loss function found in Equation equation 6. Our loss function generalizes the P3O (Wu et al., 2024) loss function, originally designed under the contextual bandit model, to the MDP setting with segments-level preferences.

Remark. We may consider variants of this loss function. For example, a KL-penalty term between π_{θ} and $\pi_{\theta_{\text{ref}}}$ would also be a reasonable choice for stability instead of clipping. This KL regularized loss function is

shown below

$$\begin{aligned} & \mathcal{L}_{\text{SP3O-KL}}(\theta_t; \theta_{t-1}, \theta_{\text{ref}}, \mathcal{D}_{\text{ref}}) \\ &= \beta D_{\text{KL}}(\pi_{\theta_t} \parallel \pi_{\theta_{\text{ref}}}) - \frac{1}{N_p} \sum_{n=1}^{N_p} D(\sigma_n^1, \sigma_n^2) \times \prod_{k=1}^L \left(\frac{\pi_{\theta_t}(a_{n,k}^1 | s_{n,k}^1)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^1 | s_{n,k}^1)} \right) \left(\frac{\pi_{\theta_{t-1}}(a_{n,k}^2 | s_{n,k}^2)}{\pi_{\theta_{\text{ref}}}(a_{n,k}^2 | s_{n,k}^2)} \right). \end{aligned}$$

In Section B.1, we present an empirical comparison of the two methods in simulated robotic control environments and show that KL regularization does not perform better than clipping in that setting. Furthermore, one may replace $\pi_{\theta_{t-1}}$ with any choice of policy π sufficiently similar to $\pi_{\theta_{\text{ref}}}$, such as $\pi_{\theta_{\text{ref}}}$ itself or a perturbed version of $\pi_{\theta_{t-1}}$. SP3O uses $\pi_{\theta_{t-1}}$ with the intuition that the loss function should have less emphasis on behaviors that have already been learned or corrected. This freedom to choose which policy to compare against is a further generalization of P3O, which uses $\theta_{t-1} = \theta_t$. In Section B.2 we present empirical results replacing $\pi_{\theta_{t-1}}$ with π_{ref} itself and show our design indeed performs better than using $\pi_{\theta_{\text{ref}}}$.

4 Theoretical Analysis

This section presents two theoretical results. The first shows that under mild assumptions, the estimator bootstrapped from segments is an unbiased estimator of the gradient of the original objective in Equation equation 7. The second quantifies the tradeoff in choosing L , the segment length.

4.1 Policy Gradient Estimation from Trajectory Segments

Given a policy π_{ref} , we construct the following discounted finite horizon MDP $\mathcal{M}'$, which we call the *segment MDP*. It models trajectory segments generated from π_{ref} as follows:

$$\mathcal{M}' = (\mathcal{S}, \mathcal{A}, d'_0, r'_h, P, \gamma, L), \text{ with } r'_h(s, a) = \begin{cases} r(s, a), & h \neq L; \\ Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s, a), & h = L. \end{cases}$$

where L is the trajectory-segment length and the initial distribution $d'_0(s)$ is the discounted occupancy measure defined as follows:

$$d'_0(s) = (1 - \gamma^L) \sum_{k=0}^{\infty} \gamma^{kL} \mathbb{P}[s_{kL+1} = s \mid s_1 \sim d_0, \pi_{\text{ref}}].$$

The following theorem shows that given a reference policy close to π_{θ} , the gradient of the expected policy value difference in $\mathcal{M}'$ is a good estimate for the gradient of the objective in $\mathcal{M}$.

Theorem 1 *If $\pi_{\text{ref}} = \pi_{\theta}$, then for any policy π' , the following holds*

$$\nabla_{\theta} J_{\mathcal{M}}(\pi_{\theta}) = \frac{1}{1 - \gamma^L} \nabla_{\theta} (J_{\mathcal{M}'}(\pi_{\theta}) - J_{\mathcal{M}'}(\pi')).$$

In other words, the gradient of the expected policy value in $\mathcal{M}$ is a scaled version of the gradient of the expected policy value difference in $\mathcal{M}'$.

The complete proof of the theorem is in Section D.4, which relies on two intermediate results. The first shows that with respect to π_{ref} , the Q function in the segment MDP $\mathcal{M}'$ equals the Q function in the original MDP $\mathcal{M}$, i.e., $Q_{\mathcal{M}'}^{\pi_{\text{ref}}}(s, a) = Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s, a)$ for all state-action pairs. This is a direct result of the definition of the terminal reward of $\mathcal{M}'$ being the Q function in $\mathcal{M}$. The second intermediate result shows that the discounted state occupancy measure of policy π_{ref} in the segment MDP $\mathcal{M}'$ is the same as the discounted state occupancy measure of π_{ref} in the original MDP $\mathcal{M}$. This is because the initial state distribution of $\mathcal{M}'$ is based on the discounted state occupancy of π_{ref} in $\mathcal{M}$. The theorem's proof then expands each side of the equation with the policy gradient theorem (Sutton et al., 1998), and then interchanges the discounted state occupancy measures and Q functions.

4.2 Trade-offs in Choosing the Segment Length

To understand the choice of segment length, we consider the segment MDP $\mathcal{M}'$ which generates trajectories of length L . Suppose that at each policy update we can collect N total timesteps of segments, mimicking an oracle that views N timesteps per update. This yields a dataset $\mathcal{D}$ of segments with size $|\mathcal{D}| = \frac{N}{2L}$. Assume $\mathcal{D} = \{\sigma^1, \sigma^2, \dots, \sigma^{|\mathcal{D}|}\}$ and each $\sigma^n = \{s_k^n, a_k^n\}_{k=1}^L$ are independently sampled. We use the sample average to estimate the expected policy value in the segment MDP $J_{\mathcal{M}'}(\pi_\theta)$ as follows:

$$\hat{J}_{\mathcal{M}'}(\pi_\theta, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \left(R(\sigma^i) + \gamma^{L-1} \hat{Q}(s_L^i, a_L^i) \right).$$

Given this, we have the following bound on the estimation error.

Proposition 1 *With probability $1 - \delta$, the estimation error is bounded as follows:*

$$\left| \hat{J}_{\mathcal{M}'}(\pi_\theta, \mathcal{D}) - J_{\mathcal{M}'}(\pi_\theta) \right| \leq \frac{1}{1-\gamma} \sqrt{\frac{L \log(2/\delta)}{N}} + \gamma^{L-1} \nu.$$

The proposition is obtained by applying Hoeffding’s inequality to the sample average estimate, and the complete proof is in Section D.5. It can be observed that if the segment length L is too short, e.g., $L = 1$, then we suffer a constant estimation error ν resulting from the inaccurate human estimation of the final state Q function. This reflects our assumption that the preference oracle has an incomplete understanding of both the policy and the environment, so they cannot tell the agent directly which action is best at each state. On the other hand, when L increases, the preference oracle error ξ gets discounted, but the variance of the partial reward increases with a smaller number of segments $|\mathcal{D}| = \lfloor \frac{N}{2L} \rfloor$. This results in a higher error in evaluating the policy from segments. The best segment length L strikes a balance between the two sources of error. In the next section, we show that the trade-off indeed appears in practice with intermediary values of L performing best in certain environments.

5 Experiments

This section compares SP3O to other reward-model-free PbRL/RLHF algorithms in a suite of robotic control tasks from the Gymnasium benchmark (Towers et al., 2024), and an LLM finetuning task. In the robotic control tasks, we compare SP3O to Online DPO (Guo et al., 2024), P3O (Wu et al., 2024), and ZPG (Zhang and Ying, 2025a). In the LLM finetuning task, we compare SP3O to Online DPO and P3O.

5.1 Robotic Control Tasks

Experiment Details. We compare SP3O with three different values of the segment length ($L = 5, 20, 50$) to Online DPO, P3O, and ZPG. Each algorithm is used to train a randomly initialized policy for 100 updates. At each policy update, all algorithms sample 10 trajectories from the environment. This yields 45 trajectory pairs for Online DPO and P3O, and 25 for ZPG. For SP3O, we randomly sample segment pairs so that the total number of timesteps in the pairs is the same as if trajectory pairs had been used. This ensures that SP3O is not giving more information to the preference oracle than Online DPO or P3O.

For each pair of segments/trajectories (σ^1, σ^2) or (τ^1, τ^2) we simulate human preference as follows. We first use the real environmental reward to compute $\mathbb{P}[\sigma^1 > \sigma^2]$ or $\mathbb{P}[\tau^1 > \tau^2]$. For SP3O, we set the $\hat{Q}$ term equal to the discounted reward sum of the remainder of the sampled rollout. For all algorithms we use the process described in Algorithm 1 with $M = 50$ to obtain an estimate $\hat{\mathbb{P}}[\sigma^1 > \sigma^2]$ or $\hat{\mathbb{P}}[\tau^1 > \tau^2]$ of the human preference. Further details are in Section C.1.

Performance Comparison. We compare each algorithm’s final policy value with the environment’s time horizon. We normalize the final policy value with the time horizon, reporting average reward per timestep. Because we are showing how performance varies with time horizon, we exclude the MuJoCo environments where termination before the horizon is reached is very common. The results, averaged across 100 seeds per point, are shown in Figure 1, where the area between the error bars represents 95% confidence intervals.

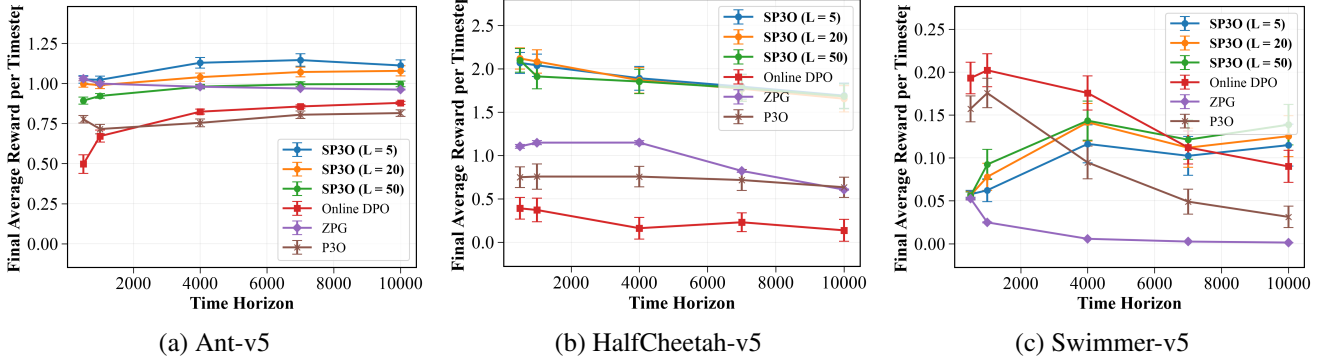


Figure 1: Final Average Reward per Timestep versus Time Horizon in MuJoCo Environments.

In Ant-v5, SP3O and Online DPO improve as the time horizon grows, P3O stays flat, and ZPG degrades. In HalfCheetah-v5, SP3O and P3O’s performance stays constant while Online DPO and ZPG’s decays. In Swimmer-v5, SP3O improves with horizon length, while Online DPO, P3O, and ZPG all degrade. Overall, this suggests that SP3O’s segment preferences become more effective as the horizon grows, while the performance of trajectory preferences in the other algorithms degrades. Furthermore, in all environments SP3O with at least one value of L outperforms the other algorithms at the longest time horizon.

Segment Length. We also empirically show the trade-off of segment length L discussed in Section 4.2. We use the same setup as the previous experiment, with the time horizon set to 1000 in HalfCheetah-v5, and vary the trajectory segment length L and the number of timesteps that can be sent to the oracle, which is equivalent to N in Proposition 1. In Figure 2 we show, for each of six oracle budgets (2^{-4} to 2^1 times the N in the main experiment), the final policy value at $L = 5, 10, \dots, 40$ relative to the seed mean, averaged over 4000 seeds. The best performing L s in each facet are starred, and the bars are 95% confidence intervals. We observe that as we increase N the best values of L increase monotonically, which is consistent with Proposition 1. A larger N shrinks the first error term $\frac{1}{1-\gamma}\sqrt{L \log(2/\delta)/N}$, so a large L can be afforded to drive down the constant error term $\gamma^{L-1}\nu$ stemming from the oracle’s estimate of the Q function.

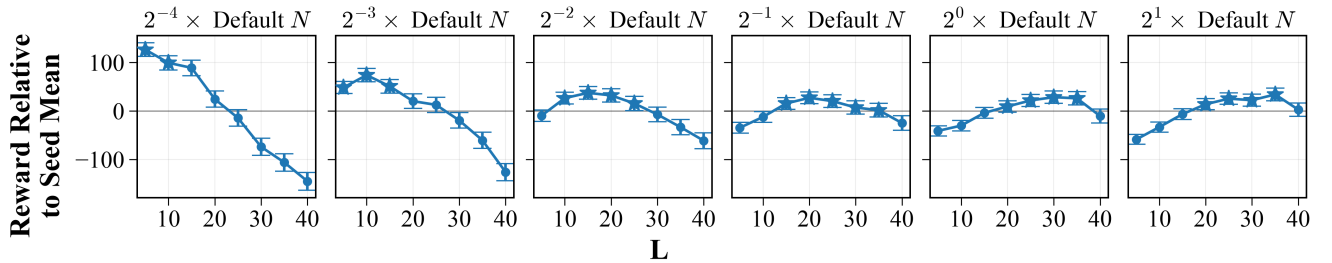


Figure 2: Ablation Study of L and N in HalfCheetah-v5.

5.2 LLM Finetuning Task

Experiment Details. In this experiment, we compare SP3O to Online DPO (Guo et al., 2024) and P3O (Wu et al., 2024) in an LLM finetuning task. We evaluate both a clipping and KL divergence regularized version of SP3O. For our base model we use GPT-J 6B (Wang and Komatsuzaki, 2021), and finetune a rank 16 LoRA (Hu et al., 2022) adapter. The goal of the task is to minimize the toxicity of the language model’s outputs while keeping the KL divergence between the fine-tuned and base models small. We use the Real Toxicity Prompts (Gehman et al., 2020) dataset to prompt the base model to have a toxic output. We measure toxicity with the pretrained Detoxify (Hanu and Unitary team, 2020) model that outputs toxicity values between 0 and 1.

During each training step, each algorithm generates 2 continuations with the current policy (with max length of 64 tokens) for each of 8 randomly selected prompts. For Online DPO and P3O these continuations are turned directly into 8 pairs. To create segments for SP3O, we split each continuation into its sentences. We then randomly pair these sentences such that each sentence is used at most once, except when there are an odd number of sentences, in which case one sentence will be in two pairs. We note that a sentence can be paired with another from the same continuation. With this setup, an evaluator will have to view a similar number of tokens at each training step regardless of whether sentences or full continuations were used. A sentence/continuation is considered preferred over another if the Detoxify score for the first sentence/continuation is at least 0.05 lower than the score for the other. All pairs that meet this threshold are then passed to the respective loss function. Further details and hyperparameters are in Section C.2.

Performance Comparison. Shown in Figure 3 are the results after running all three algorithms for 2000 steps with checkpoints evaluated every 50 steps, with 5 different seeds. At each checkpoint we report the mean toxicity on a held-out challenging subset of the prompts compared to the reverse KL divergence $D_{\text{KL}}(\pi \parallel \pi_{\text{base}})$ between the current and base models. We approximate the challenging mean toxicity versus KL curve with a monotonic PAVA regression. The shaded areas represent 95% confidence intervals, and we also show a zoomed version of this plot. We observe that both versions of SP3O consistently outperform Online DPO, and that the performance of SP3O and P3O are very similar. However, in the next section we show that growing the horizon negatively effects P3O and positively effects SP3O.

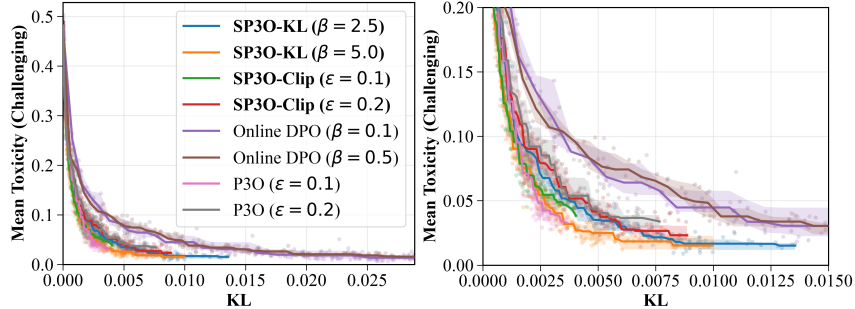


Figure 3: LLM Finetuning Results.

Continuation Length’s Effect on Performance. We also examine how the number of tokens generated in each online continuation (TPC) affects these algorithms. In this setting, TPC can be thought of as the time horizon. Shown in Figure 4 are the results when SP3O, P3O, and Online DPO are used on this task with 64, 128, and 256 tokens per continuation (TPC) averaged over 5 seeds. To make the results comparable all checkpoints were evaluated with 64 tokens per continuation.

We observe that with more tokens per continuation SP3O achieves a low toxicity score quicker. However as TPC increases, Online DPO and P3O exhibit a slower decrease in toxicity. This is despite all algorithms maintaining similar toxicity versus KL curves. As TPC increases more pairs are created for SP3O, despite the total number of tokens in the pairs being the same as if Online DPO or P3O were used. This suggests that using

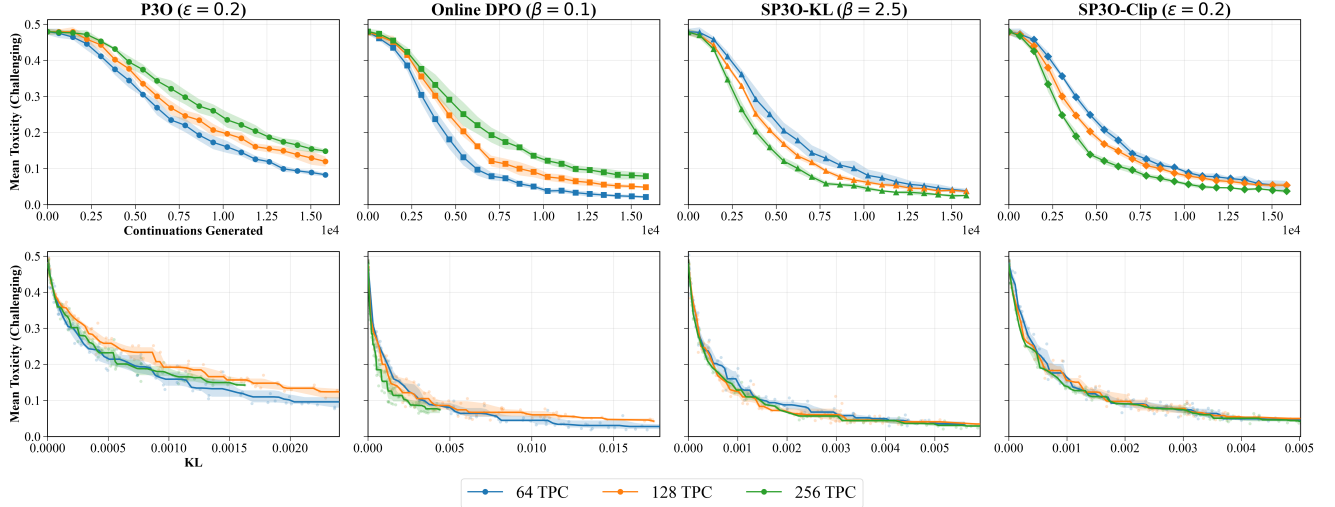


Figure 4: Comparison of Continuation Length.

segment preferences becomes more efficient than trajectory preferences as we increase the horizon length.

6 Conclusion

In this paper, we introduced SP3O, a novel reward-model-free and critic-free PbRL algorithm applicable to general MDPs with preferences based on trajectory segments. We analyzed the use of trajectory segments, showing the unbiasedness of the gradient estimator, and discovered a trade-off in segment lengths. We evaluated SP3O experimentally against other reward-model-free algorithms in a suite of robotic control tasks, and a LLM finetuning task.

Limitations and Future Directions. Like other non-reward modeling PbRL/RLHF algorithms, SP3O requires extensive preference queries throughout the entire training process. Thus, its applications are restricted to scenarios where feedback is easy to obtain. To mitigate this limitation, a rule-based oracle or AI feedback (Guo et al., 2024) could be used. A preference model such as a preference transformer (Kim et al., 2023) or an LSTM preference model (Arjona-Medina et al., 2019; Early et al., 2022) could also be used to lower the amount of human feedback required, although this approach may also suffer from issues similar to those of a reward model. Another option to avoid this limitation is to actively select queries that are likely to result in a high reward difference, such as similar strategy was used by Christiano et al. (2017), where multiple reward models were trained in parallel, and segments with high disagreement between the models would be selected for human evaluation.

AI Use Disclosure

AI tools were used in the creation of this work only for writing portions of the experimental code and final polish and proofreading of the text.

References

Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782):350–354, 2019.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.
- Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, et al. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023a.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislaw Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Borja Ibarz, Jan Leike, Tobias Pohlen, Geoffrey Irving, Shane Legg, and Dario Amodei. Reward learning from human preferences and demonstrations in atari. *Advances in neural information processing systems*, 31, 2018.
- Kimin Lee, Hao Liu, Moonkyung Ryu, Olivia Watkins, Yuqing Du, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, and Shixiang Shane Gu. Aligning text-to-image models using human feedback. *arXiv preprint arXiv:2302.12192*, 2023b.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. ISSN 00063444, 14643510.

- Qining Zhang and Lei Ying. Zeroth-order policy gradient for reinforcement learning from human feedback without reward inference. In *The Thirteenth International Conference on Learning Representations*, 2025a.
- W Bradley Knox, Stephane Hatgis-Kessell, Serena Booth, Scott Niekum, Peter Stone, and Alessandro Allievi. Models of human preference for learning reward functions. *arXiv preprint arXiv:2206.02231*, 2022.
- Joey Hejna, Rafael Rafailov, Harshit Sikchi, Chelsea Finn, Scott Niekum, W Bradley Knox, and Dorsa Sadigh. Contrastive preference learning: learning from human feedback without rl. *arXiv preprint arXiv:2310.13639*, 2023.
- Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*, 2023.
- Tianhao Wu, Banghua Zhu, Ruoyu Zhang, Zhaojin Wen, Kannan Ramchandran, and Jiantao Jiao. Pairwise proximal policy optimization: Language model alignment with comparative RL. In *First Conference on Language Modeling*, 2024.
- Qining Zhang and Lei Ying. Provable reinforcement learning from human feedback with an unknown link function. *arXiv preprint arXiv:2506.03066*, 2025b.
- Róbert Busa-Fekete, Balázs Szörényi, Paul Weng, Weiwei Cheng, and Eyke Hüllermeier. Preference-based reinforcement learning: evolutionary direct policy search using a preference-based racing algorithm. *Machine learning*, 97(3):327–351, 2014.
- Aaron Wilson, Alan Fern, and Prasad Tadepalli. A bayesian approach for policy learning from trajectory preference queries. *Advances in neural information processing systems*, 25, 2012.
- Xin Lai, Zhuotao Tian, Yukang Chen, Senqiao Yang, Xiangru Peng, and Jiaya Jia. Step-dpo: Step-wise preference optimization for long-chain reasoning of llms. *arXiv preprint arXiv:2406.18629*, 2024.
- Yiran Guo, Lijie Xu, Jie Liu, Ye Dan, and Shuang Qiu. Segment policy optimization: Effective segment-level credit assignment in RL for large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=9osvTOYbT4>.
- Shangmin Guo, Biao Zhang, Tianlin Liu, Tianqi Liu, Misha Khalman, Felipe Llinares, Alexandre Rame, Thomas Mesnard, Yao Zhao, Bilal Piot, et al. Direct language model alignment from online ai feedback. *arXiv preprint arXiv:2402.04792*, 2024.
- Dorsa Sadigh, Anca Dragan, Shankar Sastry, and Sanjit Seshia. Active preference-based learning of reward functions. 2017.
- Kimin Lee, Laura Smith, and Pieter Abbeel. Pebble: Feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training. *arXiv preprint arXiv:2106.05091*, 2021.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- Ben Wang and Aran Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. <https://github.com/kingoflolz/mesh-transformer-jax>, May 2021.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

- Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. Realtotoxicityprompts: Evaluating neural toxic degeneration in language models. *arXiv preprint arXiv:2009.11462*, 2020.
- Laura Hanu and Unitary team. Detoxify. Github. <https://github.com/unitaryai/detoxify>, 2020.
- Changyeon Kim, Jongjin Park, Jinwoo Shin, Honglak Lee, Pieter Abbeel, and Kimin Lee. Preference transformer: Modeling human preferences using transformers for rl. *arXiv preprint arXiv:2303.00957*, 2023.
- Jose A Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. Rudder: Return decomposition for delayed rewards. *Advances in Neural Information Processing Systems*, 32, 2019.
- Joseph Early, Tom Bewley, Christine Evers, and Sarvapali Ramchurn. Non-markovian reward modelling from trajectory labels via interpretable multiple instance learning. *Advances in Neural Information Processing Systems*, 35:27652–27663, 2022.
- Riad Akrou, Marc Schoenauer, and Michele Sebag. Preference-based policy learning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 12–27. Springer, 2011.
- Christian Wirth, Riad Akrou, Gerhard Neumann, and Johannes Fürnkranz. A survey of preference-based reinforcement learning methods. *Journal of Machine Learning Research*, 18(136):1–46, 2017.
- Yihan Du, Anna Winnicki, Gal Dalal, Shie Mannor, and R Srikant. Reinforcement learning with segment feedback. *arXiv preprint arXiv:2502.01876*, 2025.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Maël Franceschetti, Coline Lacoux, Ryan Ohouens, Antonin Raffin, and Olivier Sigaud. Making reinforcement learning work on swimmer. *arXiv preprint arXiv:2208.07587*, 2022.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- John Schulman. Approximating kl divergence, 2020. URL <http://joschu.net/blog/kl-approx.html>, 14, 2020.

Appendix Table of Contents

A Related Work	15
A.1 Preference-Based RL.	15
A.2 Comparison of Preference Models.	15
A.3 Use of Trajectory Segments outside of PbRL/RLHF.	16
B Additional Experiments	16
B.1 KL Regularization vs Clipping	16
B.2 Alternate Choices of Second Policy in Loss Function	16
C Implementation Details	17
C.1 Simulated Robotic Experiments.	17
C.2 LLM Finetuning Experiments.	19
D Theoretical Analysis and Proof of Theorems	21
D.1 Proof of Lemma 1	21
D.2 Proof of Lemma 2	22
D.3 Proof of Lemma 3	22
D.4 Proof of Theorem 1	22
D.5 Proof of Proposition 1	24

A Related Work

A.1 Preference-Based RL.

In the general MDP context, learning from preferences has long been studied under the umbrella of preference-based RL (PbRL) (Akrouer et al., 2011; Wilson et al., 2012; Wirth et al., 2017). In PbRL, not only trajectories and trajectory segments can be used for feedback, but also states and actions themselves (Wirth et al., 2017). Many of the methods prior to PPO+RM (Christiano et al., 2017) are based on restrictive assumptions, including discrete state/action spaces, a fully parametric policy, or a known transition kernel. This makes them inapplicable to many modern RL problems.

A.2 Comparison of Preference Models.

For two segments σ^1 and σ^2 with length L , most existing works on PbRL/RLHF (Christiano et al., 2017; Sadigh et al., 2017; Ibarz et al., 2018) assume the oracle’s preferences are based on the undiscounted return of the trajectory segment alone (partial return model), which is useful when training a reward model given the step-wise reward decomposition form. Given two segments $\sigma^1 = \{s_t^1, a_t^1\}_{t=1}^L$ and $\sigma^2 = \{s_t^2, a_t^2\}_{t=1}^L$. The preference is expressed as:

$$\mathbb{P}_{\text{partial}}[\sigma^1 > \sigma^2] = \frac{\exp\left(\sum_{t=1}^L r(s_t^1, a_t^1)\right)}{\exp\left(\sum_{t=1}^L r(s_t^1, a_t^1)\right) + \exp\left(\sum_{t=1}^L r(s_t^2, a_t^2)\right)}$$

This kind of equation is useful when training a reward model, as it can be directly estimated by the reward model for each segment pair. However, it has also been challenged (Knox et al., 2022) as not accurately representing how real human preferences are generated. Specifically, even if the partial returns of two segments are the same,

humans could still prefer one trajectory segment based on the value of the last state and action. For example, in goal-reaching tasks, an ending state closer to the goal is usually more preferred than a farther ending state even if the partial returns are the same. To accommodate this modeling flaw, Knox et al. (2022) proposed the regret-based preference model, which assumes preference oracles could make decisions based on the cumulative advantage function of each state-action pair in the segment. Specifically, Knox et al. (2022) defines the regret of a segment as:

$$\text{regret}(\sigma^1) = \sum_{t=1}^L V^{\pi^*}(s_t^1) - Q^{\pi^*}(s_t^1, a_t^1) = \sum_{t=1}^L -A^{\pi^*}(s_t^1, a_t^1),$$

where A is the advantage function. Assuming the Bradley-Terry model, we get the following equation for preference:

$$\mathbb{P}_{\text{regret}}[\sigma^1 > \sigma^2] = \frac{\exp\left(\sum_{t=1}^L -A^{\pi^*}(s_t^1, a_t^1)\right)}{\exp\left(\sum_{t=1}^L -A^{\pi^*}(s_t^1, a_t^1)\right) + \exp\left(\sum_{t=1}^L -A^{\pi^*}(s_t^2, a_t^2)\right)}.$$

The main assumption of the regret model is that the evaluators know or can make an estimate of the advantage function. This means that the human not only has an unknown latent reward function, but also has an understanding of all the environmental transition dynamics and thus can estimate this function. However, in more complicated environments, this assumption could break down.

Our model is between the partial return and regret reward models. Our model does include a version of the Q function, eliminating the main problem with the partial reward model presented by Knox et al. (2022). However, our model only assumes that the human has an understanding of the transition dynamics that arise from the policy it is evaluating. This could potentially be a much smaller subset than the knowledge of transition dynamics assumed by the regret model. Furthermore, when viewing the segments, it is possible for the human to gain this understanding and extrapolate to future states and actions. Thus, our model is able to avoid the disadvantages of the partial reward model without all the assumptions of the regret model.

A.3 Use of Trajectory Segments outside of PbRL/RLHF.

As previously discussed, trajectory segments are commonly used in PbRL/RLHF for MDPs (Christiano et al., 2017; Ibarz et al., 2018; Knox et al., 2022; Hejna et al., 2023). However, their efficacy over full-length trajectories has not been justified or theoretically examined. Outside of PbRL/RLHF, Du et al. (2025) studies a setting where trajectories can be split into segments and rewards are delayed until the ends of each segment.

B Additional Experiments

B.1 KL Regularization vs Clipping

In this section, we compare the version of SP3O that uses PPO-style clipping for stability versus a version that uses KL regularization and a version that uses no regularization. Results comparing these two versions of the algorithm are shown in Figure 5, with the time horizon set to 1000. The curves are smoothing with a sliding window of 50 updates. We observe that in Ant-v5 and HalfCheetah-v5 clipping outperforms both KL and no regularization. In Swimmer-v5, all are comparable considering the confidence intervals.

B.2 Alternate Choices of Second Policy in Loss Function

In this section, we empirically compare the version of SP3O that uses $\pi_{\theta_{t-1}}$ as the second policy in the loss function to the version that uses $\pi_{\theta_{\text{ref}}}$ as the second policy in the loss function. The training curves the two versions

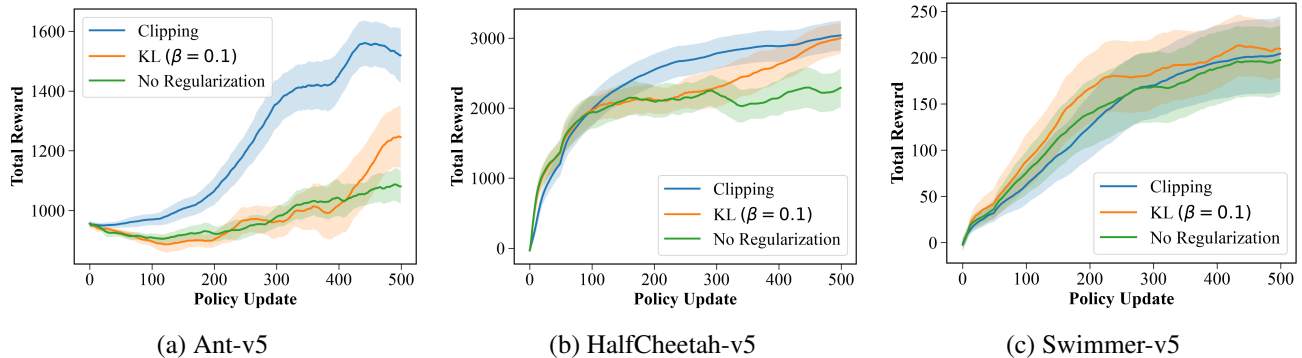


Figure 5: Ablation Study of Clipping versus KL Regularization. Shaded areas represent 95% confidence intervals.

of the algorithm are shown in Figure 6, with the time horizon set to 1000. The curves are smoothing with a sliding window of 50 updates. We observe that in Ant-v5 using $\pi_{\theta_{t-1}}$ as the second policy outperforms setting the second policy to $\pi_{\theta_{\text{ref}}}$. In the other environments, the two variants perform comparably.

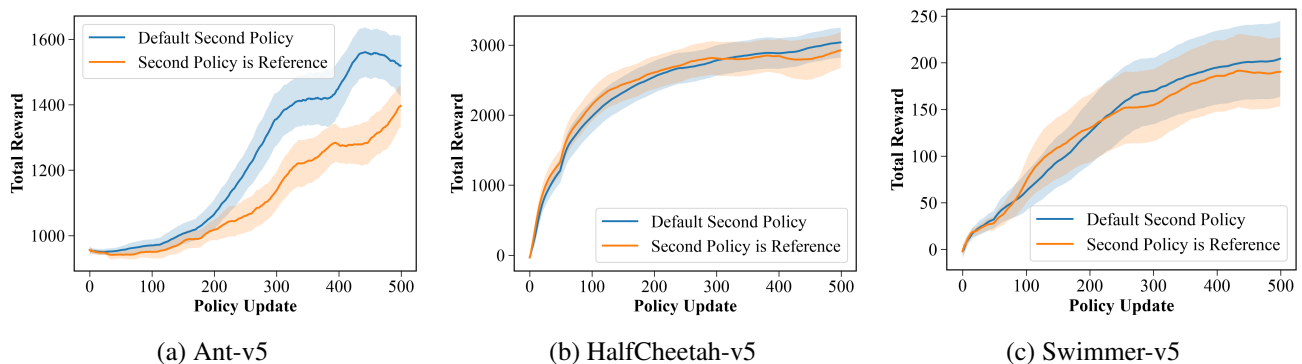


Figure 6: Ablation Study of Second Policy in Loss Function. Shaded areas represent 95% confidence intervals.

C Implementation Details

All algorithms were run in Python 3.13.3 on Ubuntu 24.04.4. The robotic experiments were run on the CPU (a Ryzen Threadripper Pro with 565 GB of RAM). The LLM experiments were run on four NVIDIA RTX PRO 6000 Blackwell Max-Qs. Python libraries and their versions are included in the code appendix.

C.1 Simulated Robotic Experiments.

All algorithms were implemented in PyTorch (Paszke et al., 2019). All policy networks had two fully connected hidden layers with 64 neurons each, and all activation functions were hyperbolic tangent (including the final layers). The Adam (Kingma and Ba, 2014) optimizer was used for all policy networks. During training, policies were Gaussian with a parameter controlling the log standard deviation for each dimension in the action space. However, during the evaluation, all policies were based on their deterministic version without the action noise.

In each algorithm, before the total reward/return difference was plugged into the respective Bradley-Terry oracle, the difference was multiplied by an expertise factor. For ZPG, Online DPO, and P3O we use $10/H$ where H is the time horizon. The expertise factor is chosen to be horizon dependent so that a unit difference in average reward between two trajectories will result in the same preference probability across all horizons. For SP3O, the

expertise factor is 0.1. This is not time dependent and higher than the algorithms because the SP3O preference model uses a discount factor, and with $\gamma = 0.99$, a unit difference in average reward will result in an increase of $\approx 1/(1 - \gamma) = 100$ in the return difference. We chose 0.1 since a unit increase in average reward results in the same preference probability when using segment of trajectory preferences.

C.1.1 Global Hyperparameters

All environments used $\gamma = 0.99$, except for Swimmer-v5, where we choose $\gamma = 0.999$ as suggested by Franceschetti et al. (2022). According to the logic above, the expertise factor for SP3O in Swimmer-v5 is 0.01. For the main experiments we used seeds 42 through 141 inclusive, and the seeds for the L ablation experiment were 42 through 4041 inclusive.

C.1.2 SP3O

Although the theoretical basis for SP3O requires sampling states from the discounted state distribution, in practice, we sample states from each timestep uniformly. This is due to simplicity, and that we are evaluating policies based on total episode reward, not discounted reward.

Our estimate of the Q function in the preference model, $\hat{Q}$, used the actual discounted return of the rest of the trajectory at the end of the segment. However, if two segments σ^1 and σ^2 were K_1 and K_2 timesteps from the horizon H . Then, for both, we would only estimate $\hat{Q}$ using the next $\min(K_1, K_2)$ timesteps.

For a segment pair (σ^1, σ^2) , the SP3O loss function will only take the gradient of the policy with respect to σ^1 . Because of this asymmetry for every segment pair (σ^1, σ^2) and estimated return difference $D(\sigma^1, \sigma^2)$, we can generate a new segment pair and return difference $(\sigma^2, \sigma^1), -D(\sigma^1, \sigma^2)$ without additional environmental interaction or queries to the preference oracle. Thus in practice for any segment pair we generate (σ^1, σ^2) we will also use the segment pair (σ^2, σ^1) .

Similar to an advantage normalization for the training of PPO, the $D(\sigma_n^1, \sigma_n^2)$ in the preference dataset are normalized to standard deviation 1 at each training step.

SP3O had all gradient norms clipped to 0.5. Further hyperparameters are shown in Table 1.

	Ant-v5	HalfCheetah-v5	Swimmer-v5
ε	0.2	0.2	0.2
Learning Rate	3e-4	3e-4	3e-4
Initial Log Std	-1.2	-1.3	0.0
Epochs	5	5	5

Table 1: SP3O Hyperparameters

C.1.3 P3O

At each policy iteration, P3O sampled 10 trajectories from the environment. This gave a total of 45 pairs per policy update. The preferences were based on a Bradley-Terry oracle using the difference in total reward sum.

Similar to an advantage normalization for the training of PPO, the reward difference terms in the preference dataset are normalized to standard deviation 1 at each training step.

P3O had all gradient norms clipped to 0.5. Further hyperparameters are shown in Table 2.

	Ant-v5	HalfCheetah-v5	Swimmer-v5
ε	0.2	0.2	0.2
Initial Log Std	-1.2	-1.3	0.0
Learning Rate	3e-4	3e-4	3e-4
Epochs	5	5	5

Table 2: P3O Hyperparameters

C.1.4 Online DPO

At each policy iteration, Online DPO sampled 10 trajectories from the environment. This gave a total of 45 pairs per policy update. The preferences were based on a Bradley-Terry oracle using the difference in total reward sum. Online DPO had all gradient norms clipped to 0.5. Further hyperparameters are shown in Table 3.

	Ant-v5	HalfCheetah-v5	Swimmer-v5
β	0.1	0.1	0.1
Initial Log Std	-1.2	-1.3	0.0
Learning Rate	7e-4	7e-4	7e-4
Epochs	5	5	5

Table 3: Online DPO Hyperparameters

C.1.5 ZPG

At each policy iteration, ZPG sampled 10 trajectories from the environment. This gave a total of 25 pairs per policy update. This value is different from that of Online DPO and P3O, since half of ZPG’s trajectories are generated using the perturbed policy. The learning rate was 0.003, the perturbation distance was 0.1, and there was one epoch per policy update. Our implementation of ZPG also used an Adam (Kingma and Ba, 2014) style momentum system at each update step.

C.2 LLM Finetuning Experiments.

C.2.1 Hyperparameters

LoRA adapters were applied to all linear layers with rank 16, scaling $\alpha = 32$, and dropout 0.05.

The AdamW (Loshchilov and Hutter, 2017) optimizer was used for all algorithms with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. The global gradient norm is clipped to 1.0.

Shown in Figure 7 are the toxicity versus KL curves of each algorithm with different learning rates. For the regularization parameters in these experiments we use $\beta = 1.0$ for SP3O-KL, $\varepsilon = 0.2$ for SP3O-Clip, $\varepsilon = 0.2$ for P3O, and $\beta = 0.5$ for DPO.

In P3O, SP3O-KL, and SP3O-Clip, we observe that the curve does not improve when the learning rate goes lower than $1e-5$. In DPO, we observe that the curve does not improve when the learning rate goes lower than $2e-5$. These are the learning rate values we use for the main experiments.

The KL divergence is calculated by randomly picking 1000 prompts from the entire dataset. For all KL divergences we use the k_3 estimator (Schulman, 2020). All log probabilities of sentences and full continuations are calculated with a mean reduction to avoid issues with varying lengths. The seeds for all experiments were 42, 43, 44, 45, 46.

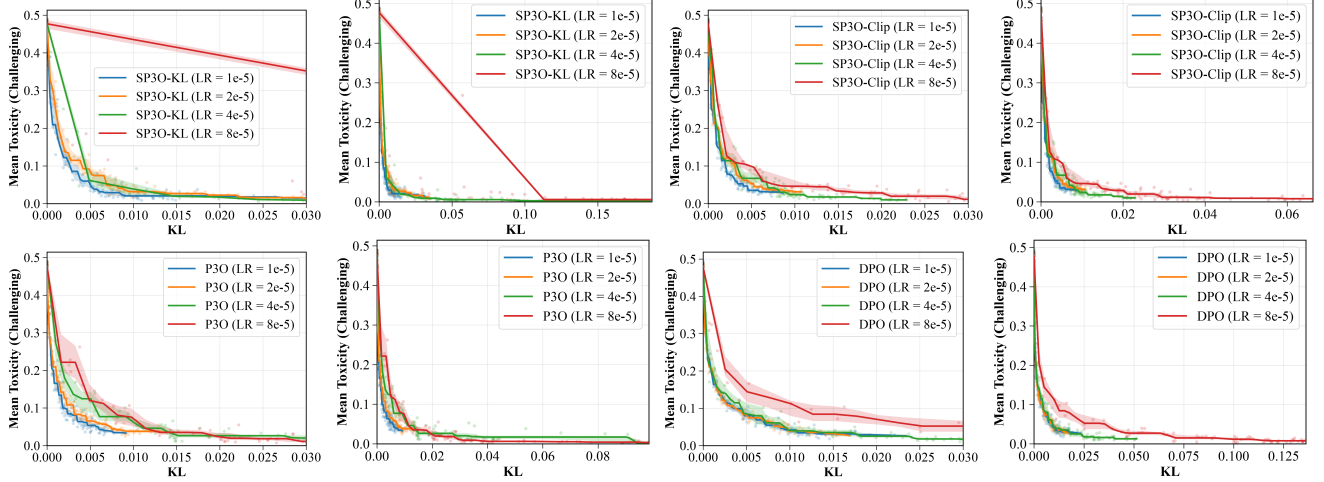


Figure 7: Learning Rate Tuning Results.

In SP3O and P3O, we set the reward difference terms to $+1$ or -1 depending on which segment/continuation is preferred, and don't infer any magnitude of the preference. In P3O, we use the P3O-V1 (Clipping Separately) from the original P3O paper (Wu et al., 2024).

In all algorithms, we did not update the reference policy in the loss functions from the base model, despite sampling from the current policy. This is because the difference between the models is small, and keeping the base model as the reference gives better regularization. In Figure 8, we provide a comparison on updating the reference policy to the current generating policy versus keeping the base policy as the reference in the loss function. We observe that for all algorithms keeping the reference policy as the base model provides a better curve.

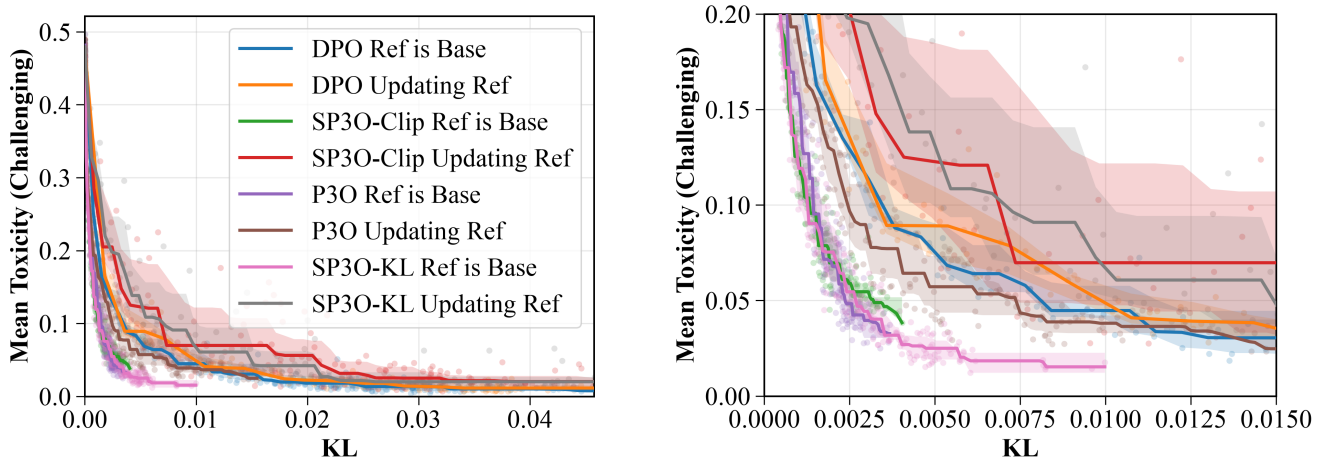


Figure 8: Comparison of Changing Reference Policy.

C.2.2 Fitting Toxicity vs KL Curves

We fit these curves with the Pool-Adjacent-Violators Algorithm (PAVA), using the cloud of points generated by each training runs checkpoints. For the 95% confidence intervals are found through bootstrapping 300 times.

C.2.3 SP3O Sentence Splitting

We split a continuation at points where a period, question mark, or exclamation mark appears and at the end of the continuation. We note that these punctuation marks can show up within a sentence such as in the title "Dr.". This creates a small disadvantage for SP3O, as the created sentences could be very short. However, this kind of situation is rare enough such that the impact on SP3O's performance should be negligible.

D Theoretical Analysis and Proof of Theorems

D.1 Proof of Lemma 1

Lemma 1 Suppose we have three policies π_1 , π_2 and π_{ref} . We assume that the support of π_1 and π_2 is enclosed within the support of π_{ref} , that is $\pi_1(s|a) > 0$ or $\pi_2(s|a) > 0$ implies $\pi_{\text{ref}}(s|a) > 0$, $\forall s \in \mathcal{S}, a \in \mathcal{A}$. Then we can estimate the objective difference of π_1 and π_2 using only π_{ref} as follows.

Proof 1 Firstly, we can estimate the objective using only the first H timesteps due to the discount factor. Specifically, the error between two trajectories is:

$$\left| \sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) - \sum_{t=1}^H \gamma^{t-1} r(s_t, a_t) \right| = \gamma^H \sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) \leq \gamma^H \sum_{t=1}^{\infty} \gamma^{t-1} = \gamma^H \frac{1}{1-\gamma}.$$

Given this the error in estimating the expected value difference surrogate will be at most $\frac{2\gamma^H}{1-\gamma}$, which can be very small given H is the effective horizon. Then we can estimate the difference in expected value as:

$$\begin{aligned} J_{\mathcal{M}}(\pi_1) - J_{\mathcal{M}}(\pi_2) &= \mathbb{E}_{a_t^1 \sim \pi_1(\cdot|s_t^1), s_1^1 \sim d_0} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t^1, a_t^1) \right] - \mathbb{E}_{a_t^2 \sim \pi_2(\cdot|s_t^2), s_1^2 \sim d_0} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t^2, a_t^2) \right] + \mathcal{O}(\gamma^H) \\ &= \mathbb{E}_{\substack{a_t^1 \sim \pi_1(\cdot|s_t^1), s_1^1 \sim d_0, \\ a_t^2 \sim \pi_2(\cdot|s_t^2), s_1^2 \sim d_0}} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t^1, a_t^1) - \sum_{t=1}^H \gamma^{t-1} r(s_t^2, a_t^2) \right] + \mathcal{O}(\gamma^H) \\ &= \mathbb{E}_{\substack{a_t^1 \sim \pi_{\text{ref}}(\cdot|s_t^1), s_1^1 \sim d_0, \\ a_t^2 \sim \pi_{\text{ref}}(\cdot|s_t^2), s_1^2 \sim d_0}} \left[\frac{\mathbb{P}_{\pi_1}(\tau^1) \mathbb{P}_{\pi_2}(\tau^2)}{\mathbb{P}_{\pi_{\text{ref}}}(\tau^1) \mathbb{P}_{\pi_{\text{ref}}}(\tau^2)} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t, a_t) - \sum_{t=1}^H \gamma^{t-1} r(s'_t, a'_t) \right] \right] + \mathcal{O}(\gamma^H), \end{aligned}$$

where $\tau^1 = \{(s_t^1, a_t^1)\}_{t=1}^H$ and $\tau^2 = \{(s_t^2, a_t^2)\}_{t=1}^H$.

Now, let us analyze each importance sampling term as follows:

$$\begin{aligned} \frac{\mathbb{P}_{\pi_1}(\tau^1)}{\mathbb{P}_{\pi_{\text{ref}}}(\tau^1)} &= \frac{d_0(s_1^1) \pi_1(a_1^1|s_1^1) P(s_2^1|s_1^1, a_1^1) \cdots P(s_H^1|s_{H-1}^1, a_{H-1}^1) \pi_1(a_H^1|s_H^1)}{d_0(s_1^1) \pi_{\text{ref}}(a_1^1|s_1^1) P(s_2^1|s_1^1, a_1^1) \cdots P(s_H^1|s_{H-1}^1, a_{H-1}^1) \pi_{\text{ref}}(a_H^1|s_H^1)} \\ &= \prod_{h=1}^H \frac{\pi_1(a_h^1|s_h^1)}{\pi_{\text{ref}}(a_h^1|s_h^1)}, \end{aligned}$$

where the last equality is due to the same transition in each trajectory. Therefore, we can express the expected value difference as:

$$\begin{aligned} J_{\mathcal{M}}(\pi_1) - J_{\mathcal{M}}(\pi_2) &= \mathbb{E}_{\substack{a_t^1 \sim \pi_{\text{ref}}(\cdot|s_t^1), s_1^1 \sim d_0, \\ a_t^2 \sim \pi_{\text{ref}}(\cdot|s_t^2), s_1^2 \sim d_0}} \left[\prod_{t=1}^H \frac{\pi_1(a_t|s_t)}{\pi_{\text{ref}}(a_t|s_t)} \frac{\pi_2(a_t|s_t)}{\pi_{\text{ref}}(a_t|s_t)} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t^1, a_t^1) - \sum_{t=1}^H \gamma^{t-1} r(s_t^2, a_t^2) \right] \right] + \mathcal{O}(\gamma^H). \end{aligned}$$

D.2 Proof of Lemma 2

Lemma 2 For all $s \in \mathcal{S}$ and $a \in \mathcal{A}$, we have $Q_{\mathcal{M}'}^{\pi_{\text{ref}}}(s, a) = Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s, a)$.

Proof 2 Given some $s_1 \in \mathcal{S}, a_1 \in \mathcal{A}$. If s_1 is one step from termination then it follows from definitions that $Q_{\mathcal{M}'}^{\pi_{\text{ref}}}(s_1, a_1) = r'(s_1, a_1) = Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_1, a_1)$. In the other case, assume without loss of generality that s_1 is $k > 1$ steps from termination in $\mathcal{M}'$. Then:

$$\begin{aligned} Q_{\mathcal{M}'}^{\pi_{\text{ref}}}(s_1, a_1) &= \mathbb{E}_{\pi_{\text{ref}}} \left[\sum_{t=1}^k \gamma^{t-1} r'(s_t, a_t) \right] \\ &= \sum_{t=1}^k \gamma^{t-1} \mathbb{E}_{\pi_{\text{ref}}} [r'(s_t, a_t)] \\ &= \gamma^{k-1} \mathbb{E}_{\pi_{\text{ref}}} [Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_k, a_k)] + \sum_{t=1}^{k-1} \gamma^{t-1} \mathbb{E}_{\pi_{\text{ref}}} [r(s_t, a_t)] \\ &= Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_1, a_1). \end{aligned}$$

which concludes the proof. ■

D.3 Proof of Lemma 3

Lemma 3 For any $s \in \mathcal{S}$, we have $d_{\mathcal{M}'}^{\pi_{\text{ref}}}(s) = d_{\mathcal{M}}^{\pi_{\text{ref}}}(s)$.

Proof 3 Given some $s \in \mathcal{S}$, by the definition of the discounted occupancy defined, we have:

$$\begin{aligned} d_{\mathcal{M}'}^{\pi_{\text{ref}}}(s) &= \frac{1-\gamma}{1-\gamma^L} \sum_{l=1}^L \gamma^{l-1} \mathbb{P}[s_l = s \mid \pi_{\text{ref}}, s_1 \sim d'_0] \\ &= \frac{1-\gamma}{1-\gamma^L} \sum_{l=1}^L \sum_{t=1, L+1, 2L+1, \dots}^L \left(\gamma^{t-1} (1-\gamma^L) \gamma^{l-1} \mathbb{P}[s_{l+t-1} = s \mid \pi_{\text{ref}}, s_t \sim d_{\mathcal{M}, t}^{\pi_{\text{ref}}}] \right) \\ &= (1-\gamma) \sum_{t=1, L+1, 2L+1, \dots}^L \sum_{l=1}^L \gamma^{t+l-2} \sum_{s' \in \mathcal{S}} \mathbb{P}[s_{l+t-1} = s \mid \pi_{\text{ref}}, s_t = s'] \mathbb{P}[s_t = s' \mid \pi_{\text{ref}}, s_1 \sim d_0] \\ &= (1-\gamma) \sum_{t=1, L+1, 2L+1, \dots}^L \sum_{l=1}^L \left(\gamma^{t+l-2} \mathbb{P}[s_{l+t-1} = s \mid \pi_{\text{ref}}, s_1 \sim d_0] \right) \\ &= (1-\gamma) \sum_{t=1}^{\infty} \gamma^{t-1} \mathbb{P}[s_t = s \mid \pi_{\text{ref}}, s_1 \sim d_0] \\ &= d_{\mathcal{M}}^{\pi_{\text{ref}}}(s). \end{aligned}$$

which concludes the proof. ■

D.4 Proof of Theorem 1

Theorem 1 If we choose $\pi_{\text{ref}} = \pi_{\theta}$, the gradient of the expected policy value in $\mathcal{M}$ is a scaled version of the gradient of the expected policy value difference in $\mathcal{M}'$, i.e., for an arbitrary policy π' , we have

$$\nabla_{\theta} J_{\mathcal{M}}(\pi_{\theta}) = \frac{1}{1-\gamma^L} \nabla_{\theta} (J_{\mathcal{M}'}(\pi_{\theta}) - J_{\mathcal{M}'}(\pi')).$$

Proof 4 Firstly the $\nabla_\theta(J_{\mathcal{M}'}(\pi_\theta) - J_{\mathcal{M}'}(\pi'))$ simplifies to $\nabla_\theta J_{\mathcal{M}'}(\pi_\theta)$. Now we will expand $\nabla_\theta J_{\mathcal{M}'}(\pi_\theta)$ in a similar style to the Policy Gradient Theorem (Sutton et al., 1998).

$$\begin{aligned}\nabla_\theta J_{\mathcal{M}'}(\pi_\theta) &= \nabla_\theta \mathbb{E}_{s_1 \sim d'_0} [V_{\mathcal{M}'}^{\pi_\theta}(s_1)] \\ &= \sum_{s_1 \in \mathcal{S}} \nabla_\theta(d'_0(s_1) V_{\mathcal{M}'}^{\pi_\theta}(s_1)) \\ &= \sum_{s_1 \in \mathcal{S}} (\nabla_\theta d'_0(s_1)) V_{\mathcal{M}'}^{\pi_\theta}(s_1) + d'_0(s_1) \nabla_\theta V_{\mathcal{M}'}^{\pi_\theta}(s_1).\end{aligned}$$

Since d'_0 doesn't depend on θ we find that this equals:

$$\begin{aligned}\nabla_\theta J_{\mathcal{M}'}(\pi_\theta) &= \sum_{s_1 \in \mathcal{S}} d'_0(s_1) \nabla_\theta V_{\mathcal{M}'}^{\pi_\theta}(s_1) \\ &= \mathbb{E}_{s_1 \sim d'_0} [\nabla_\theta V_{\mathcal{M}'}^{\pi_\theta}(s_1)] \\ &= \mathbb{E}_{s_1 \sim d'_0} \left[\nabla_\theta \sum_{a_1 \in \mathcal{A}} \pi_\theta(a_1 | s_1) Q_{\mathcal{M}'}^{\pi_\theta}(s_1, a_1) \right] \\ &= \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta \ln \pi_\theta(a_1 | s_1) Q_{\mathcal{M}'}^{\pi_\theta}(s_1, a_1)] + \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta Q_{\mathcal{M}'}^{\pi_\theta}(s_1, a_1)] \\ &= \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta \ln \pi_\theta(a_1 | s_1) Q_{\mathcal{M}'}^{\pi_\theta}(s_1, a_1)] + \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta r'(s_1, a_1)] \\ &\quad + \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta \gamma V_{\mathcal{M}'}^{\pi_\theta}(s_2)].\end{aligned}$$

Since when $k < L$, $r'_k(s, a) = r(s, a)$ we get:

$$\begin{aligned}\nabla_\theta J_{\mathcal{M}'}(\pi_\theta) &= \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta \ln \pi_\theta(a_1 | s_1) Q_{\mathcal{M}'}^{\pi_\theta}(s_1, a_1)] + \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta r(s_1, a_1)] \\ &\quad + \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta \gamma V_{\mathcal{M}'}^{\pi_\theta}(s_2)] \\ &= \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta \ln \pi_\theta(a_1 | s_1) Q_{\mathcal{M}'}^{\pi_\theta}(s_1, a_1)] + \gamma \mathbb{E}_{s_1 \sim d'_0, a_1 \sim \pi_\theta(\cdot | s_1)} [\nabla_\theta V_{\mathcal{M}'}^{\pi_\theta}(s_2)].\end{aligned}$$

Then we continue to recursively evaluate the expected value of $\nabla_\theta V_{\mathcal{M}'}^{\pi_\theta}(s_t)$ until we reach the final step:

$$\begin{aligned}&\nabla_\theta J_{\mathcal{M}'}(\pi_\theta) \\ &= \sum_{t=1}^{L-1} \gamma^{t-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t | s_t) Q_{\mathcal{M}'}^{\pi_\theta}(s_t, a_t)] + \gamma^{L-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_L | s_L) Q_{\mathcal{M}'}^{\pi_\theta}(s_L, a_L)] \\ &= \sum_{t=1}^{L-1} \gamma^{t-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t | s_t) Q_{\mathcal{M}'}^{\pi_\theta}(s_t, a_t)] + \gamma^{L-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_L | s_L) r'_L(s_L, a_L)].\end{aligned}$$

Then since $r'_L(s, a) = Q_{\mathcal{M}'}^{\pi_{\text{ref}}}(s, a)$ we get:

$$\begin{aligned}&\nabla_\theta J_{\mathcal{M}'}(\pi_\theta) \\ &= \sum_{t=1}^{L-1} \gamma^{t-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t | s_t) Q_{\mathcal{M}'}^{\pi_\theta}(s_t, a_t)] + \gamma^{L-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_L | s_L) Q_{\mathcal{M}'}^{\pi_{\text{ref}}}(s_L, a_L)].\end{aligned}$$

Since we chose $\pi_{\text{ref}} = \pi_\theta$ we get:

$$\begin{aligned}\nabla_\theta J_{\mathcal{M}'}(\pi_\theta) &= \sum_{t=1}^L \gamma^{t-1} \mathbb{E}_{s_1 \sim d'_0, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t | s_t) Q_{\mathcal{M}'}^{\pi_\theta}(s_t, a_t)] \\ &= \frac{1 - \gamma^L}{1 - \gamma} \mathbb{E}_{s \sim d_{\mathcal{M}'}^{\pi_\theta}, \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t | s_t) Q_{\mathcal{M}'}^{\pi_\theta}(s_t, a_t)].\end{aligned}$$

Combining this with Lemmas 2 and 3 we get:

$$\nabla_{\theta} J_{\mathcal{M}'}(\pi_{\theta}) = \frac{1 - \gamma^L}{1 - \gamma} \mathbb{E}_{s \sim d_{\mathcal{M}', \pi_{\theta}}^{\pi_{\theta}}} [\nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) Q_{\mathcal{M}}^{\pi_{\theta}}(s_t, a_t)].$$

Moving to the LHS of the original equation, the policy gradient theorem (Sutton et al., 1998) tells us:

$$\nabla_{\theta} J_{\mathcal{M}}(\pi_{\theta}) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d_{\mathcal{M}}^{\pi_{\theta}}} [\mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [Q_{\mathcal{M}}^{\pi_{\theta}}(s, a) \nabla_{\theta} \ln \pi_{\theta}(a | s)]]$$

Note that what we simplified $\nabla_{\theta}(J_{\mathcal{M}'}(\pi_{\theta}) - J_{\mathcal{M}'}(\pi'))$ too is simply a scalar multiple of $\nabla_{\theta} J_{\mathcal{M}}(\pi_{\theta})$. Thus:

$$\nabla_{\theta} J_{\mathcal{M}}(\pi_{\theta}) = \frac{1}{1 - \gamma^L} \nabla_{\theta}(J_{\mathcal{M}'}(\pi_{\theta}) - J_{\mathcal{M}'}(\pi')),$$

which concludes the proof. ■

D.5 Proof of Proposition 1

Proposition 1 With probability $1 - \delta$, the estimation error is bounded as follows:

$$\left| \hat{J}_{\mathcal{M}'}(\pi_{\theta}, \mathcal{D}) - J_{\mathcal{M}'}(\pi_{\theta}) \right| \leq \frac{1}{1 - \gamma} \sqrt{\frac{L \log(2/\delta)}{N}} + \gamma^{L-1} \nu.$$

Proof 5 Recall that the estimator $\hat{J}_{\mathcal{M}'}(\pi_{\theta}, \mathcal{D})$ consists of two terms:

$$\begin{aligned} \hat{J}_{\mathcal{M}'}(\pi_{\theta}, \mathcal{D}) &= \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} R(\sigma^i) + \gamma^{L-1} \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \hat{Q}(s_L^i, a_L^i) \\ &= \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} (R(\sigma^i) + \gamma^{L-1} Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_L^i, a_L^i)) \\ &\quad + \gamma^{L-1} \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \xi_i. \end{aligned}$$

For the first term, since the trajectory segments σ^i are sampled independently, we can use Hoeffding's bound as follows. Notice that the reward is bounded in $[0, 1]$, the Q function is bounded in $[0, 1/(1 - \gamma)]$, and $|\mathcal{D}| = N/2L$, then with probability $1 - \delta$, we have:

$$\begin{aligned} &\left| \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} (R(\sigma^i) + \gamma^{L-1} Q_{\mathcal{M}}^{\pi_{\text{ref}}}(s_L^i, a_L^i)) - J_{\mathcal{M}'}(\pi_{\theta}) \right| \\ &\leq \left(\sum_{k=1}^{L-1} \gamma^{k-1} + \gamma^{L-1} \frac{1}{1 - \gamma} \right) \sqrt{\frac{\log(2/\delta)}{2|\mathcal{D}|}} \\ &= \left(\frac{1 - \gamma^{L-1}}{1 - \gamma} + \frac{\gamma^{L-1}}{1 - \gamma} \right) \sqrt{\frac{L \log(2/\delta)}{N}} \\ &= \frac{1}{1 - \gamma} \sqrt{\frac{L \log(2/\delta)}{N}}. \end{aligned}$$

The errors ξ_i are adversarial and thus bounded by:

$$\gamma^{L-1} \frac{1}{|D|} \sum_{i=1}^{|D|} \xi_i \leq \gamma^{L-1} \nu.$$

So we obtain the final bound as:

$$\left| \hat{J}_{\mathcal{M}'}(\pi_\theta, \mathcal{D}) - J_{\mathcal{M}'}(\pi_\theta) \right| \leq \frac{1}{1-\gamma} \sqrt{\frac{L \log(2/\delta)}{N}} + \gamma^{L-1} \nu,$$

which concludes the proof. ■